

CMPE412 – System Programming

Comprehensive Code Summary

Processes & Threads in C on Linux

Topics Covered:

fork() · wait() · sleep() · Signals · Pipes · Shared Memory
Threads · Race Conditions · Mutex · Semaphores

—— PROCESSES IN C ——

This file (Processes_code.pdf) is part of the CMPE412 – System Programming course and covers process creation, synchronization, inter-process communication, and process control using the C language on a Linux system. All examples rely on POSIX system calls available through standard Linux headers.

Overview of Key System Calls

fork()	Creates a child process. Returns 0 in the child, the child's PID in the parent.
getpid()	Returns the PID of the calling process.
getppid()	Returns the PID of the parent process.
wait() / wait(NULL)	Makes the parent block until a child process terminates.
sleep(n)	Suspends the calling process for n seconds.
usleep(μs)	Suspends the calling process for a given number of microseconds.
kill(pid, sig)	Sends a signal to a process. Used for SIGSTOP, SIGCONT, SIGKILL.
pipe(fd[2])	Creates a unidirectional communication channel between two processes.
read() / write()	Read from / write to a file descriptor (including pipe ends).
shm_open()	Creates or opens a POSIX shared memory object.
mmap()	Maps a shared memory object into the process address space.
munmap() / shm_unlink()	Unmaps and removes a shared memory object.

Example 1 – Creating a Child Process with fork()

Purpose: Demonstrates the fundamental use of fork() to split a single process into a parent and a child. After the call, both processes execute independently from the same point in code.

How it works:

- fork() is called once but returns twice — once in each process.
- If the return value (pid) is 0, the code is running inside the child process.
- If pid > 0, it is the parent process and pid holds the child's actual PID.
- getpid() returns the PID of the current process in either branch.

```

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    int pid = fork();    // Splits into two processes at this line

    if (pid == 0) {      // This block runs ONLY in the child process
        printf("Hello from Child!\n");
        printf("process ID: %d, fork return: %d\n", getpid(), pid);
    } else {             // This block runs ONLY in the parent process
        printf("Hello from Parent!\n");
        printf("process ID: %d, fork return: %d\n", getpid(), pid);
    }
    return 0;
}

```

 **Note:** The fork return value inside the child is always 0, but getpid() still shows the child's actual OS-assigned PID. Inside the parent, fork() returns the child's PID, and getpid() shows the parent's PID.

Example 2 – Enforcing Execution Order with wait()

Purpose: Demonstrates why wait() is essential when you need the child process to complete before the parent continues. Without it, the operating system scheduler can run either process in any order, making output non-deterministic.

Scenario: Print 1–5 in the child, then 6–10 in the parent.

- The child is assigned $n = 1$ (prints 1 to 5).
- The parent is assigned $n = 6$ (prints 6 to 10).
- wait() in the parent blocks it from printing until the child finishes.

```

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    int pid = fork();
    int n;

```

```

if (pid == 0)  n = 1;    // Child starts at 1
else          n = 6;    // Parent starts at 6

if (pid != 0)          // Only the parent waits
    wait();            // Blocks parent until child finishes

for (int i = n; i < n + 5; i++) {
    printf("%d ", i);   // Each process prints its 5 numbers
}

printf("\n");
return 0;
}

```

 **Note:** Removing `wait()` causes both processes to race, and the output order (1-5 vs 6-10) becomes unpredictable. `wait()` is the standard synchronization tool for parent-child relationships.

Example 3 – `sleep()` and Zombie/Orphan Processes

Purpose: Illustrates what happens when the parent process finishes before the child. When `sleep()` is applied to the child, the parent exits first — at that moment the child becomes an orphan and is automatically re-parented to the `init` process (PID 1).

- The child calls `sleep(1)`, causing it to pause for 1 second.
- The parent finishes immediately and exits.
- When the child eventually wakes up and calls `getppid()`, it will return 1 (or the `init` PID), not the original parent's PID.

```

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    int pid = fork();

    if (pid == 0) {
        sleep(1);    // Child sleeps – parent likely exits first
    }
}

```

```
// Both processes print this line
printf("Parent ID: %d, Process ID: %d\n", getppid(), getpid());
return 0;
}
```

 **Note:** When the parent exits first, the child becomes an orphan. The OS assigns it a new parent (init/systemd). This is why `getppid()` in the child returns 1 after a delay — its true parent no longer exists.

Example 4 – Controlling a Process with Signals (SIGSTOP, SIGCONT, SIGKILL)

Purpose: Shows how the parent process can control the execution lifecycle of a child process using POSIX signals. The child runs an infinite loop and the parent dictates when it runs, pauses, and is killed.

Signal Summary:

- SIGSTOP — Suspends a process immediately (cannot be caught or ignored).
- SIGCONT — Resumes a suspended process.
- SIGKILL — Terminates a process immediately (cannot be caught or ignored).

Flow of the Program:

- Child: runs an infinite loop printing its status with a 50ms delay (usleep).
- Parent: immediately sends SIGSTOP to pause the child, then asks the user for a duration.
- If duration > 0: sends SIGCONT, sleeps for that duration, then re-sends SIGSTOP.
- If duration ≤ 0: sends SIGKILL to terminate the child, then wait() cleans up.

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>    // Required for kill() and signal constants

int main() {
    int pid = fork();

    if (pid == 0) {
        // CHILD: runs forever, printing its status every ~50ms
        while (1) {
            printf("Child process running\n");
```

```

        usleep(50000); // 50,000 microseconds = 50ms
    }
    } else {
        kill(pid, SIGSTOP); // Immediately stop the child
        int t;

        do {
            printf("Time in seconds for execution: ");
            scanf("%d", &t);

            if (t > 0) {
                kill(pid, SIGCONT); // Resume child
                sleep(t);           // Let it run for t seconds
                kill(pid, SIGSTOP); // Pause child again
            }
        } while (t > 0);

        kill(pid, SIGKILL); // Terminate child
        wait(NULL);         // Reap zombie — clean up child resources
    }

    return 0;
}

```

 **Note:** wait(NULL) after SIGKILL is critical. Without it, the terminated child becomes a zombie process — it has exited but its entry in the process table is not freed until the parent reads its exit status.

Example 5 – One-Way Communication Between Processes Using Pipes

Purpose: Introduces pipes as a mechanism for inter-process communication (IPC). A pipe has two ends: fd[0] for reading and fd[1] for writing. Data flows in one direction only.

Key Rules for Using Pipes:

- Always close the unused end in each process. If the reading end is left open in the writer process, the OS cannot detect end-of-file.
- The child writes data into the pipe, and the parent reads from it.
- close() must be called on both ends after use to prevent resource leaks.

Part A – Simple value transfer from child to parent:

```
#include <stdio.h>
```

```

#include <sys/types.h>
#include <unistd.h>

int main() {
    int fd[2];          // fd[0] = read end, fd[1] = write end
    pipe(fd);           // Create the pipe
    int pid = fork();

    if (pid == 0) {      // CHILD: writer
        close(fd[0]);    // Close unused read end
        int x;
        printf("Input a number: ");
        scanf("%d", &x);
        write(fd[1], &x, sizeof(int)); // Write x into pipe
        close(fd[1]);
    } else {             // PARENT: reader
        close(fd[1]);    // Close unused write end
        int y;
        read(fd[0], &y, sizeof(int)); // Read x from pipe into y
        close(fd[0]);
        printf("Value from child: %d\n", y);
    }
    return 0;
}

```

Part B – Parallel array summation using pipes:

The child computes the sum of the first half of an array, and the parent sums the second half. The child sends its partial sum through the pipe. The parent reads it and adds both halves together for the final total.

```

// Array: {1, 2, 3, 5, 4, 3}, N = 6
// Child → sums arr[0..2] = 1+2+3 = 6
// Parent → sums arr[3..5] = 5+4+3 = 12
// Child sends 6 through pipe → Parent computes 6 + 12 = 18

if (pid == 0) {
    start = 0; end = N / 2;          // Child: first half
} else {
    start = N / 2; end = N;          // Parent: second half
}

int sum = 0;
for (int i = start; i < end; i++) sum += arr[i];

if (pid == 0) {

```

```

    close(fd[0]);
    write(fd[1], &sum, sizeof(int)); // Child sends its partial sum
    close(fd[1]);
} else {
    int childSum;
    close(fd[1]);
    read(fd[0], &childSum, sizeof(int)); // Parent receives child sum
    close(fd[0]);
    printf("Total sum: %d\n", sum + childSum);
    wait(NULL);
}

```

Example 6 – Two-Way Communication Using Two Pipes

Purpose: Demonstrates bidirectional IPC between a parent and child. Using a single pipe for two-way communication is unreliable because both processes share the same read and write ends. The correct solution is to use two separate pipes.

Single-Pipe Problem (shown first):

- Using one pipe, the parent writes a value and the child reads it, then the child writes back and the parent reads.
- This is fragile — removing even one `printf` may cause the program to deadlock (get stuck waiting).
- This happens because both ends share the same buffer, causing race conditions on reads and writes.

Two-Pipe Solution (correct approach):

- p1: dedicated for child → parent communication (child writes, parent reads).
- p2: dedicated for parent → child communication (parent writes, child reads).
- Each process closes the ends it does not use, ensuring clean, non-blocking I/O.

```

int p1[2], p2[2];
pipe(p1); // p1: child writes → parent reads
pipe(p2); // p2: parent writes → child reads
int pid = fork();

if (pid == 0) { // CHILD
    close(p1[0]); // Not reading from p1
    close(p2[1]); // Not writing to p2

```



```

    int x;

    read(p2[0], &x, sizeof(x));    // Read value sent by parent
    printf("Child received: %d\n", x);
    x *= 4;                        // Process the value (multiply by 4)
    write(p1[1], &x, sizeof(x));    // Send result back to parent
    printf("Child sent: %d\n", x);

    close(p1[1]); close(p2[0]);

} else {                          // PARENT
    close(p1[1]);                  // Not writing to p1
    close(p2[0]);                  // Not reading from p2

    int y = rand() % 10;           // Generate a random number 0-9
    write(p2[1], &y, sizeof(y));    // Send to child
    printf("Parent sent: %d\n", y);
    read(p1[0], &y, sizeof(y));    // Receive result from child
    printf("Result from child: %d\n", y);
    close(p1[0]); close(p2[1]);
}

```

 **Note:** With two pipes, each direction of communication is fully independent. This eliminates deadlocks caused by both processes trying to read/write the same buffer simultaneously.

Example 7 – Shared Memory Between Processes

Purpose: Introduces POSIX shared memory as an alternative IPC mechanism. Unlike pipes, shared memory allows both processes to directly read and write the same memory region — making it significantly faster for large data transfers.

Steps to Use Shared Memory:

- `shm_open()` — Creates or opens a named shared memory object (like a file name for memory).
- `ftruncate()` — Sets the size of the shared memory region.
- `mmap()` — Maps the shared memory into the process's virtual address space so it can be accessed like a pointer.
- `fork()` — Both parent and child inherit the mapping and can read/write the same memory.
- `munmap()` — Unmaps the memory from the process's address space.
- `shm_unlink()` — Deletes the shared memory object from the OS (cleanup).

```

#include <stdio.h>
#include <unistd.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <sys/wait.h>
#define SHM_NAME "/my_shm"

int main() {
    int shm_fd;
    int *shared_int;

    // Step 1: Create the shared memory object
    shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(int)); // Step 2: Set size to one int

    // Step 3: Map it into both processes' address space
    shared_int = mmap(0, sizeof(int), PROT_READ | PROT_WRITE,
                      MAP_SHARED, shm_fd, 0);

    pid_t pid = fork();

    if (pid == 0) { // CHILD: reads then modifies the value
        sleep(2); // Wait for parent to write first
        printf("Child: read value = %d\n", *shared_int); // Reads 100
        *shared_int = 200; // Overwrites to 200
    } else { // PARENT: writes then reads after child
        *shared_int = 100;
        printf("Parent: initialized to %d\n", *shared_int);
        wait(NULL); // Wait for child to finish
        printf("Parent: after child = %d\n", *shared_int); // Reads 200

        // Cleanup
        munmap(shared_int, sizeof(int));
        shm_unlink(SHM_NAME);
    }

    return 0;
}

```

 **Note:** Shared memory is the fastest IPC mechanism since there is no kernel buffer copying — both processes directly access the same physical memory. However, it requires careful synchronization (e.g., via semaphores) to avoid race conditions when both processes access the shared region simultaneously.

— THREADS IN C —

This file (Threads_code.pdf) is part of CMPE412 – System Programming (Assignment II) and covers thread creation, identification, synchronization, and safe concurrent access. All examples use the POSIX thread library (pthreads) and must be compiled with the -lpthread flag.

Compilation command:

```
gcc file.c -o output -lpthread
```

Overview of Key pthread Functions

pthread_create()	Creates a new thread. Takes a thread variable, attributes, function pointer, and argument.
pthread_join()	Blocks the calling thread until the specified thread finishes. Equivalent to wait() for processes.
pthread_self()	Returns the ID of the calling thread (type pthread_t).
pthread_mutex_init()	Initializes a mutex variable. Must be called before lock/unlock.
pthread_mutex_lock()	Acquires the mutex. If already locked, blocks until it becomes available.
pthread_mutex_trylock()	Tries to acquire the mutex without blocking. Returns 0 on success, non-zero if unavailable.
pthread_mutex_unlock()	Releases the mutex so other threads can acquire it.
pthread_mutex_destroy()	Frees the mutex resources after use.
sem_init()	Initializes a semaphore with a given initial counter value.
sem_wait()	Decrements the semaphore. Blocks if the counter is 0 (like lock).
sem_post()	Increments the semaphore, releasing one waiting thread (like signal/unlock).
sem_destroy()	Frees semaphore resources.

Example 1 – Creating a Single Thread

Purpose: Introduces the most basic use of pthreads. A thread is a lightweight execution unit within the same process. Unlike fork(), threads share the same memory space as the creating process.

- pthread_t t1 declares a thread variable (a handle to the thread).

- `pthread_create(&t1, NULL, &routine, NULL)` creates the thread and runs the routine function inside it.
- `pthread_join(t1, NULL)` makes `main()` wait for the thread to complete before exiting.

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>    // Always include this for thread support

// Thread function – must return void* and accept void*
void* routine(void *arg) {
    printf("Testing thread\n");
    return NULL;
}

int main() {
    pthread_t t1;                // Declare a thread handle
    pthread_create(&t1, NULL, &routine, NULL); // Create and start thread
    pthread_join(t1, NULL);      // Wait for thread to finish
    return 0;
}
```

Examples 2 & 3 – Creating Multiple Threads and Getting Thread IDs

Purpose: Shows how to launch multiple threads simultaneously, and how to identify each thread using `pthread_self()`. All threads in the same process share the same PID (`getpid()`), but each has its own unique thread ID.

Creating two threads running the same routine:

```
void* routine(void* arg) {
    printf("Testing thread\n");
    sleep(3);                // Both threads sleep concurrently – total wait is ~3s not 6s
    printf("Ending thread\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, routine, NULL); // Start thread 1
    pthread_create(&t2, NULL, routine, NULL); // Start thread 2
    pthread_join(t1, NULL); // Wait for thread 1
    pthread_join(t2, NULL); // Wait for thread 2
    return 0;
}
```

```
}
```

Using `pthread_self()` to identify threads:

```
void* routine() {  
    pthread_t tid = pthread_self(); // Get this thread's unique ID  
    printf("Thread ID: %ld in PID: %d\n", tid, getpid());  
    // Note: all threads print the same PID — they share the process!  
    sleep(3);  
    printf("Ending thread\n");  
}
```

Creating 10 threads with a loop:

```
int main() {  
    pthread_t t[10];  
  
    // First loop: create all threads  
    for (int i = 0; i < 10; i++) {  
        pthread_create(&t[i], NULL, &routine, NULL);  
    }  
  
    // Second loop: join all threads  
    // IMPORTANT: joining in the same loop as creation forces threads  
    // to run sequentially instead of in parallel!  
    for (int i = 0; i < 10; i++) {  
        pthread_join(t[i], NULL);  
    }  
    return 0;  
}
```

 **Note:** If `pthread_create` and `pthread_join` are placed in the same loop, thread `i` must finish before thread `i+1` is even created — eliminating parallelism. Always separate creation and joining into two distinct loops to allow true concurrent execution.

Example 4 – Race Condition

Purpose: Demonstrates one of the most critical concurrency bugs — the race condition. When two threads read, modify, and write a shared variable without synchronization, the final result is unpredictable and often incorrect.

Why it happens — `N++` is not atomic:

The statement `N++` appears as a single instruction but actually consists of three CPU operations:

- 1. READ the current value of `N` from memory into a CPU register.
- 2. INCREMENT the register value by 1.
- 3. WRITE the result back to `N` in memory.

When two threads interleave these steps, both can read the same old value, increment it, and write back the same new value — effectively losing one of the increments.

```
int N = 0;    // Shared global variable – both threads access this

void* routine() {
    int M = 10;
    for (int i = 0; i < M; i++) {
        N++;  // NOT thread-safe! Read-modify-write can be interrupted
    }
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, &routine, NULL);
    pthread_create(&t2, NULL, &routine, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    // Expected: 20, but may print 11, 14, 17... it varies every run
    printf("Value of N: %d", N);
    return 0;
}
```

 **Note:** With `M=10`, the race is unlikely to manifest because threads finish too fast. With `M=1,000,000` the problem becomes obvious — you will consistently see `N` values well below 2,000,000. The OS scheduler interrupts threads mid-operation, causing lost updates.

Example 5 – Solving Race Conditions with Mutex

Purpose: Demonstrates how a mutex (mutual exclusion lock) prevents race conditions by ensuring only one thread at a time can execute a critical section — the portion of code that accesses shared data.

How Mutex Works:

- `pthread_mutex_lock(&mt)` — The thread attempts to acquire the lock. If another thread holds it, this call blocks until it is released.
- `pthread_mutex_unlock(&mt)` — Releases the lock so another waiting thread can acquire it.
- Only one thread can hold the lock at any given time, serializing access to `N++`.

```
int N = 0;
pthread_mutex_t mt;    // Global mutex - shared between threads

void* routine() {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&mt);    // Acquire lock - block others
        N++;                        // Critical section: now thread-safe
        pthread_mutex_unlock(&mt);  // Release lock - let others in
    }
}

int main() {
    pthread_mutex_init(&mt, NULL); // Initialize before use
    pthread_t t1, t2;

    if (pthread_create(&t1, NULL, &routine, NULL) != 0) return 1;
    pthread_create(&t2, NULL, &routine, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Value of N: %d", N); // Always 2,000,000 now
    pthread_mutex_destroy(&mt);  // Free mutex resources
    return 0;
}
```

trylock vs lock:

- `pthread_mutex_lock()` — Blocks indefinitely until the lock is available.
- `pthread_mutex_trylock()` — Returns immediately. Returns 0 if lock was acquired, non-zero if the mutex was already locked.
- `trylock` is useful when a thread should do something else instead of waiting (non-blocking pattern).

```
// Example: trylock usage pattern
if (pthread_mutex_trylock(&mt) == 0) {
    printf("Mutex locked - entering critical section\n");
    N++;
}
```

```
pthread_mutex_unlock(&mt);
} else {
    printf("Mutex denied - mutex is busy, skipping\n");
}
```

 **Note:** Using mutex makes the program correct but slower. Each lock/unlock pair is a system call with overhead. For 50,000,000 iterations, this can be noticeably slower than an unsynchronized version — but correctness always takes priority over speed.

Example 6 – Semaphore for Controlling Thread Concurrency

Purpose: Introduces semaphores as a more flexible synchronization primitive than mutex. While a mutex allows exactly 1 thread at a time, a semaphore allows a configurable number of threads (N) to be active concurrently.

Semaphore vs Mutex:

Mutex	Binary — either locked (0) or unlocked (1). Only 1 thread at a time.
Semaphore (value=1)	Equivalent to a mutex when initialized to 1.
Semaphore (value=N)	Allows up to N threads to enter the critical section simultaneously.

How sem_wait and sem_post work:

- `sem_wait(&S)` — Checks the counter. If > 0 , decrements it and continues. If $= 0$, blocks the thread.
- `sem_post(&S)` — Increments the counter, waking one blocked thread if any are waiting.

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h> // Required for sem_t and sem functions

sem_t S; // Global semaphore

void* routine() {
    sem_wait(&S); // Acquire semaphore - blocks if counter = 0
    printf("Thread processing...\n");
    sleep(3); // Simulate work
    printf("Ending Thread\n");
    sem_post(&S); // Release semaphore - increments counter
}
```



```

int main() {
    sem_init(&S, 0, 1); // Initialize: 0=process-local, 1=max 1 thread at a time

    pthread_t t[10];
    for (int i = 0; i < 10; i++)
        pthread_create(&t[i], NULL, &routine, NULL);
    for (int i = 0; i < 10; i++)
        pthread_join(t[i], NULL);

    sem_destroy(&S); // Free semaphore resources
    return 0;
}

```

With `sem_init(&S, 0, 1)`, only 1 thread runs at a time. Changing it to `sem_init(&S, 0, 3)` allows 3 threads to execute simultaneously — which a binary mutex cannot achieve.

 **Note:** Semaphores can replace mutexes (with value=1) but offer more power. Use a mutex when you need strict binary mutual exclusion. Use a semaphore when you want to limit concurrency to N threads or implement producer-consumer patterns.

Processes vs Threads – Key Differences

Aspect	Processes (fork)	Threads (pthread)
Memory Space	Separate (copy of parent)	Shared within same process
Creation Cost	High (full memory copy)	Low (lightweight)
Communication	IPC (pipes, shared memory)	Shared variables directly
Crash Isolation	Child crash doesn't affect parent	One thread crash can kill all
Synchronization	wait(), signals	mutex, semaphore, join
Identifier	PID (getpid)	Thread ID (pthread_self)
Use Case	Isolated tasks, security	Parallel tasks sharing data

Compilation Quick Reference

Basic process code	<code>gcc file.c -o output && ./output</code>
Thread code (pthreads)	<code>gcc file.c -o output -lpthread && ./output</code>
Shared memory code	<code>gcc file.c -o output -lrt && ./output</code>
Check GCC version	<code>gcc --version</code>
Install GCC	<code>sudo apt install gcc</code>

End of Summary — CMPE412 System Programming